NORTHERN UNIVERSITY
OF BUSINESS & TECHNOLOGY KHULNA
Knowledge for an Enlightening Future

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Requirements Specification (SRS)

PROJECT ON

AI Programming Tutor & Bug Explainer

Course Code: CSE 4104

Course Title: Software Development III

SUBMITTED TO

Name: Md. Riaz Mahmud

Assistant Professor

Department of CSE

Northern University of Business & Technology, Khulna.

SUBMITTED BY

Team Name	CSE4104 - 7B - T04
Section	7B
Project Title	AI Programming Tutor & Bug Explainer
Member Names	M.M. Asfy Or Rayhan Alisha Rahman Nadia Afrin Fatima Tanjim Rafa
Student IDs	11230121106 11230121112 11230121121 11230321556
GitHub Repository	https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer

Date of Submission: 08/06/2026

Contents

1. **Introduction** 1

2. **Functional Requirement**.....6

3. **Non-Functional Requirements**.....13

4. **User Roles**.....16

5. **Use Cases**.....18

Software Requirement Specification (SRS)

for

AI Programming Tutor & Bug Explainer

1. Introduction

1.1 Project Title

AI Programming Tutor & Bug Explainer.

1.2 Project Overview

AI Programming Tutor & Bug Explainer is a web-based, AI-powered programming tutor and bug explainer platform built specifically for beginner-level programmers. The primary goal of this system is to bridge the gap between a student's confusion and their understanding acting as a patient, always-available digital teacher who explains things in simple language, including Bangla and English together, so that learners feel comfortable regardless of their background. The platform is not just a simple chatbot. It is a full-stack application with a React (Vite) frontend styled using Tailwind CSS, connected to a Node.js and Express backend API. The backend communicates with multiple AI providers Groq, Google Gemini, and OpenAI through a centralized AI service layer, so that the provider can be swapped at any time by changing a single environment variable without touching any application logic. All data such as user accounts, chat history, XP progress, and AI memory notes are stored in a MongoDB database. What makes AI Programming Tutor & Bug Explainer different from a plain chatbot is its memory system. For logged-in students, the platform quietly keeps notes about what topics they have worked on, which errors they have encountered, and where they tend to struggle. The next time they ask a question, the AI tutor uses this memory to give a more personalized and contextually relevant response something a plain chatbot cannot do. The system also includes a gamification layer with XP points, levels, streaks, and an activity dashboard to keep students motivated over time.

1.3 Problem Statement

Learning to program is one of the most valuable skills a student can develop in the modern world, yet it remains disproportionately difficult for beginners in Bangladesh and similar contexts. The problem is not that the concepts themselves are beyond reach it is that the resources available to most beginner students are built for an audience that already has a foundation the beginner does not yet possess. Textbooks explain recursion by referencing call stacks. Online documentation describes functions by referencing other functions. Stack Overflow answers assume the reader already understands what went wrong. For a first-year student encountering these resources alone, the experience is frequently one of compounding confusion rather than progressive understanding. The language barrier compounds this problem significantly. The overwhelming majority of programming tutorials, documentation, and error message explanations are written in technical English. For students who think and reason more naturally in Bangla, working through a confusing concept becomes doubly difficult when they must also wrestle with the language in which the explanation is written. No widely available tool addresses this specifically there is no platform designed to explain programming concepts or compiler errors in Bangla or in a natural Bangla-English mix that reflects how these students actually communicate. Compiler and runtime error messages represent a particularly acute version of this problem. When a beginner's program fails to compile or crashes at runtime, the error message they receive "Segmentation fault (core dumped)", "NullPointerException at line 14", "TypeError: 'NoneType' object is not subscriptable" is written entirely in machine-level technical language that gives them almost no actionable guidance. A student who does not know what a null pointer is, what a segmentation fault means at the operating system level, or what it means for a variable to have no type cannot make progress from that error message alone. They either ask a friend, wait for their instructor, or give up. In a classroom setting where one instructor serves dozens of students, the wait is often long enough to break concentration and motivation entirely. Beyond concept explanation and error handling, beginner students face the problem of not knowing whether the code they write even when it works is written well. Working code and good code are not the same thing, and students who never receive feedback on their code quality develop habits that create increasingly serious problems as their programs grow larger. Variable names like 'a', 'b', and 'x'; nested loops where a single-pass algorithm would suffice; functions that do five things at once these patterns are invisible to a

student who has no experienced reviewer to consult. Professional code review is a service that only students with access to senior mentors or paid tutoring can realistically obtain. The same accessibility gap applies to understanding real-world code. Open-source repositories on GitHub represent an enormous library of practical programming knowledge, but they are effectively inaccessible to beginners who do not know how to orient themselves within an unfamiliar codebase. Without guidance on which files to read first, what the project is trying to do, and how its components connect, a student who opens a repository for the first time is confronted with dozens or hundreds of files and no map. Similarly, lecture notes and textbook pages even when a student has access to them often contain explanations that were written at a pace and level of abstraction that outstrips what a first-year student can absorb independently. Existing AI tools such as general-purpose chatbots do address some of these problems in isolation, but they introduce their own limitations for this use case. A general-purpose chatbot has no memory of the student's past interactions every session begins from scratch, meaning the student must re-explain their context every time and the AI cannot adapt its responses based on what the student has already worked on or where they consistently struggle. General-purpose chatbots are also not designed or prompted for the specific task of beginner programming education: their responses can be too long, too technical, too English-centric, or structured in a way that does not match how a confused student needs information presented to them. They also lack the accountability and engagement features XP, levels, streaks, personal history that help students maintain the habit of consistent practice over weeks and months rather than using the tool once and forgetting it. AI Programming Tutor & Bug Explainer is built as a direct response to all of these gaps. It provides a single, integrated platform where a beginner programmer in Bangladesh can get concept explanations in their natural language, understand their error messages immediately without waiting for a human instructor, receive constructive feedback on working code, explore real codebases at their own pace, and build a consistent study habit supported by a memory system that makes every subsequent interaction more relevant than the last. The system is designed to run entirely for free using the Groq API's free tier or a local mock mode so that cost is not a barrier to access for any student who needs it.

1.4 Objectives

The core objective of AI Programming Tutor & Bug Explainer is to make programming education

accessible, affordable, and encouraging for students who are just starting out. Most beginners struggle not because the concepts are impossible, but because the explanations they find are too technical, too fast, or written assuming prior knowledge that they do not have yet. AI Programming Tutor & Bug Explainer addresses this by generating responses that are specifically designed for first-year students using analogies from everyday life, mixing Bangla with English when it helps, and always providing a short working code example alongside the explanation. Beyond concept explanation, the platform aims to help students debug their own code by clearly identifying what is wrong and why not just fixing it silently, but explaining the root cause so the student learns to avoid the same mistake in the future. Similarly, the Compiler Error Translator feature turns scary-looking error messages such as *"Segmentation Fault"* or *"NullPointerException"* into plain, human-readable sentences that a beginner can actually understand and act on. The platform also aims to support self-directed learning by allowing students to upload their lecture PDFs, textbook pages, or even photos of handwritten notes and receive AI explanations of that content. For students who want to explore open-source code, the GitHub Repository Explainer lets them paste any public repository URL and receive a plain-language breakdown of the entire project along with file-by-file explanations. The Practice Problem Generator creates custom coding challenges based on any topic and difficulty level, complete with hints and a commented solution, so students can test and reinforce their understanding at their own pace.

1.5 Scope

AI Programming Tutor & Bug Explainer is scoped as a web application accessible from any modern browser on both desktop and mobile devices. The system supports two types of users guests and registered users where guests can use all AI tools freely without creating an account, and registered users additionally benefit from saved history, XP tracking, and AI memory personalization. A third role, Admin, provides platform oversight capabilities including user management and activity monitoring. The current version of the platform includes seven fully functional AI tools: the AI Tutor, Bug Explainer, Compiler Error Translator, Code Improver, GitHub Repository Explainer, PDF and Image Document Explainer, and Practice Problem Generator. It also includes user authentication with JWT-based security, a gamified personal dashboard, a Voice Tutor feature using the browser's built-in speech engine, and a complete Admin Panel. Features that are intentionally left out of this version but planned for future releases include

live code execution in a sandboxed environment, a real-time AI mock interview module, a daily challenge and leaderboard system, personalized learning roadmaps driven by the Weak Topic model, a Redis caching layer for repeated prompts, and integration with a billing system for a paid subscription tier. These are mapped in the project roadmap and are designed to slot into the existing architecture without requiring structural changes.

2. Functional Requirements

2.1 User Registration

Any visitor to the platform can create a new account by providing their full name, a valid email address, and a password of at least six characters. When the registration form is submitted, the backend validates that all three fields are present and non-empty. It then checks the database to confirm that no existing account is already registered under that email address if a duplicate is found, the server returns a 409 Conflict error with a clear message so the user knows to log in instead. If the email is new, the system hashes the password using the bcryptjs library with a salt factor of ten before storing anything in the database. The raw password is never written to disk at any point. Once the user document is created in MongoDB, the server automatically checks whether the registered email appears in the ADMIN_EMAILS environment variable list. If it does, the new account is immediately created with the admin role rather than the default student role. This mechanism allows the first administrator to be set up simply by configuring an environment variable before deployment, without any manual database editing. After successful registration, the server returns the new user's profile object with the password field stripped out along with a signed JWT token that the client stores and uses to authenticate all future requests.

2.2 User Login

Registered users can log in by submitting their email address and password. The backend retrieves the corresponding user document from MongoDB specifically requesting the password field which is excluded from normal queries by default and uses bcryptjs to compare the submitted plain-text password against the stored hash. If either the email does not exist or the password comparison fails, the server returns a 401 Unauthorized response with a generic 'Invalid email or password' message, intentionally avoiding specifics that could help an attacker. When credentials are valid, the system performs an additional check: if the user's email appears in the current ADMIN_EMAILS list but their stored role is not yet 'admin', the role is updated in the database before responding. This ensures that an administrator who was added to the env list after initial registration is automatically upgraded on their next login without any manual intervention. The

server then signs a new JWT token using the server-side secret and returns it to the client along with the full user profile.

2.3 AI Tutor

The AI Tutor is the heart of the platform. A user whether logged in or visiting as a guest can type any programming concept or question into the input field and receive a structured, beginner-friendly explanation. Before sending the request to the AI, the backend checks whether the user is authenticated and, if so, retrieves their memory context from the Memory collection. This context is a compact summary of what the student has been working on recently and which topics they tend to struggle with, built from notes that the system adds after every interaction. The memory context is prepended to the AI prompt invisibly, instructing the model to personalize its response for example, if the student has previously asked about loops and now asks about recursion, the AI might reference loops as a familiar concept to build the analogy. The AI is always instructed to structure its tutor response in four parts: a one-line simple definition, a short everyday analogy, a small working code example, and a sentence of encouragement. The tone is intentionally warm and patient, never condescending, and the language mixes Bangla and English based on how the student writes their question. After a successful response, the system saves the interaction to the ChatHistory collection with a feature tag of 'tutor' and awards the logged-in user five XP points. If the database is unavailable, this history-saving step is skipped silently and the AI response is still returned to the student without interruption.

2.4 Bug Explainer

The Bug Explainer allows a student to paste any piece of code written in any programming language into the Monaco editor and receive a detailed, educational explanation of what is wrong with it. The student can optionally specify the programming language to help the AI give more targeted feedback, but this field is not required. The AI does not simply rewrite the code silently; it is specifically prompted to teach, not just fix. The AI response is structured in four sections. First, it gives a one-line plain-language summary of what the bug actually is. Second, it walks through the specific problematic lines and explains why they are wrong for example, explaining that an off-by-one error in a loop condition causes the array index to go out of bounds on the last

iteration, and why that matters. Third, it presents the corrected code in a clean, syntax-highlighted code block. Fourth, it gives one or two short prevention tips so the student understands how to avoid the same class of mistake in the future. Each bug explanation awards ten XP to a logged-in user and saves the interaction under the 'bugExplainer' feature tag. The system also adds a short memory note such as 'Debugged some Python code' to help the AI tutor personalize future responses.

2.5 Compiler Error Translator

One of the most frustrating experiences for a beginner programmer is encountering an error message that reads like a foreign language. A message like 'Segmentation fault (core dumped)' or 'Exception in thread main java.lang.NullPointerException' gives very little guidance to someone who is new to programming. The Compiler Error Translator is designed specifically to solve this problem. The student pastes the exact error message they received whether from a compiler, a runtime environment, or an interpreter into the input field and optionally selects the programming language the error came from. The AI is prompted to respond with four structured pieces of information: first, what the error is actually saying in simple words; second, the most common reasons why this error typically occurs; third, concrete steps the student can take to fix it; and fourth, a short illustrative example if one would be helpful. The response is always written in plain Bangla and English mixed together, so students who are more comfortable in Bangla can follow along without confusion. This feature is the flagship tool of the platform the one most specifically designed to reduce the intimidation factor of learning to program. It awards five XP per use and saves the interaction under the 'errorTranslator' feature tag.

2.6 Code Improver

Once a student has working code, the Code Improver helps them level up by identifying ways to make that code cleaner, more readable, and more efficient. The student pastes their code into the editor and the AI reviews it with the perspective of a senior developer explaining things to a junior. The AI is explicitly instructed not to rewrite the code into something unrecognizable the goal is to teach improvements the student can actually understand and apply themselves. The response covers three areas. First, it acknowledges what the student did well, which is important for

confidence and motivation. Second, it lists specific suggestions such as using more descriptive variable names, removing unnecessary nested loops, applying standard naming conventions, or replacing a brute-force approach with a more efficient data structure like a hashmap. Each suggestion is accompanied by a plain-language explanation of why it is better for example, 'nested loop er bodole hashmap use korle onek faster hobe karon lookup time O(1)'. Third, where relevant, the AI provides a short improved code snippet to illustrate the changes. The Code Improver awards ten XP per use and saves the interaction under the 'codeImprover' feature tag.

2.7 GitHub Repository Explainer

The GitHub Repository Explainer allows students to explore and understand any public open-source project without needing to clone it or read through potentially hundreds of files on their own. The student pastes a GitHub repository URL and selects their preferred output language from three options: Bangla, Banglish, or English. The backend then calls the GitHub REST API to fetch the repository's metadata, README file, and complete file tree. This information is compiled into a structured prompt that asks the AI to explain the project in five parts: a one-line summary of what the project is; two to three lines about what it does and who it is for; the main programming languages and frameworks it uses; two to four key files that a newcomer should read first, with a one-line explanation of each file's role; and a brief guess at how to run the project locally based on what is visible in the README. The AI is strictly instructed not to invent files or features that are not actually present in the data provided to it. After the overview is displayed, the student sees an interactive file tree rendered in the browser. Clicking on any file name triggers a second API call to fetch that specific file's content from GitHub, which is then sent to the AI for a dedicated file-level explanation. This explanation walks through the file's purpose, its important sections and functions, and any tricky concepts it introduces. The GitHub API works without authentication for public repositories at up to sixty requests per hour per IP address; a GitHub personal access token can be configured in the environment to raise this to five thousand requests per hour.

2.8 PDF and Image Document Explainer

The Document Explainer supports two distinct input types: PDF files and images. For PDF uploads, the backend uses the unpdf library to extract the raw text content from the uploaded file

without any external dependencies. This extracted text is then passed to the AI along with an explanation prompt that instructs the model to identify what the document is about, explain its key ideas in beginner-friendly terms, highlight any code present and describe what it does, and flag any obvious bugs if code is found. For image uploads such as a smartphone photo of a whiteboard, handwritten notes, or a printed textbook page the system takes a different path. The image is encoded as base64 and sent to a vision-capable AI model: Google Gemini 1.5 Flash, OpenAI GPT-4o-mini, or the Groq Llama-4 Scout vision model, depending on which provider is configured. The vision model reads the visual content of the image and generates the same style of beginner-friendly explanation. This makes it possible for a student to take a photo of their professor's board notes after class and immediately receive a plain-language breakdown of what was written. In both cases, the student can select the output language before uploading, and they can also type a specific question about the document in which case the AI answers that question first before giving the general explanation. This feature is particularly useful for students who are working through lecture materials independently or studying for exams.

2.9 Practice Problem Generator

The Practice Problem Generator allows a student to request a custom coding challenge on any topic they choose, at one of several difficulty levels. The student types the topic for example 'arrays', 'recursion', or 'binary search' and selects a difficulty, and the system sends this to the AI with a strict instruction to respond exclusively in a specific JSON format with no additional text, no markdown wrapping, and no backticks. The JSON structure returned by the AI contains seven fields: a title for the problem, the problem statement written in beginner-friendly language, a sample input, the expected sample output, an array of two or three progressive hints that guide the student toward the solution without giving it away, the actual solution in commented code, and a plain-language explanation of how the solution works. The backend parses this JSON defensively if the AI returns something that cannot be parsed, the raw text is wrapped in a minimal fallback object so the student always sees something useful rather than an error. This feature is designed for self-directed practice and is particularly valuable for students preparing for assessments or interviews. The practice problem endpoint already exists in the backend at this stage; a dedicated practice arena UI page is listed in the roadmap for a future version.

2.10 Voice Tutor

Every AI response displayed on the platform whether from the Tutor, Bug Explainer, Error Translator, or any other tool can be read aloud to the student by clicking the Voice button. This feature uses the Web Speech API that is built directly into modern browsers, meaning no external service, no API key, and no additional cost of any kind. The speech synthesis plays the response text in the same language the response was written in, making it especially helpful for students who prefer listening over reading or who want to review content while doing something else. The Voice Tutor is available to all users without any restriction it is not gated behind a premium subscription and is equally accessible to guests and registered students alike. This was a deliberate design decision to ensure that the most accessible learning format is also the most universally available one.

2.11 AI Memory and Personalization

The memory system is what separates AI Programming Tutor & Bug Explainer from a generic chatbot. Every time a logged-in student completes an interaction with any AI tool, the system automatically generates a short summary note for example, 'Asked the tutor about recursion' or 'Debugged some C code' or 'Hit a Python IndexError' and stores it in the Memory collection alongside the user's ID. Only the forty most recent notes are kept, so the memory stays compact and relevant rather than growing without bound. Before generating any AI response for a logged-in user, the system retrieves the user's last six memory notes and their top five weak topics from the WeakTopic collection, ranked by how many times each topic has caused a mistake. These are combined into a compact context paragraph that is prepended to the AI prompt invisibly. The AI is instructed to use this context to personalize its response naturally for example, referencing something the student worked on recently when building an analogy but not to list the memory notes out loud or make the student feel observed. This creates the experience of talking to a tutor who actually remembers you, rather than starting fresh every time.

2.12 Dashboard and Gamification

Logged-in students have access to a personal Dashboard that gives them a live view of their progress on the platform. The dashboard displays their current XP total, calculated level based on

the formula Level equals the floor of XP divided by one hundred plus one, their current daily activity streak, and a Recharts-powered activity chart showing their recent engagement over time. Below the chart, a history section lists their last fifty interactions across all AI tools, each with the feature name, topic or code snippet, and timestamp. The gamification system awards XP for every meaningful interaction: five points for using the AI Tutor or Error Translator, and ten points for using the Bug Explainer or Code Improver. These awards happen automatically after a successful AI response and are applied directly to the user's account in MongoDB. If the database write fails for any reason, the failure is caught silently and the AI response is still returned to the student the system always prioritizes the learning experience over the tracking layer.

2.13 Admin Panel

The Admin Panel is a restricted area accessible only to users with the admin role. It provides platform-wide visibility that a student account cannot see. When an admin visits the panel, the backend performs four parallel database queries: a count of all registered users, a count of admin accounts specifically, a total count of all ChatHistory records across the entire platform, and an aggregation that groups ChatHistory records by feature tag to show how many times each AI tool has been used. It also retrieves the five users with the highest XP scores. This data is returned in a single API response and rendered as statistics cards and a usage chart on the frontend. The admin can also view the complete registered user list, showing each user's name, email, role, XP, level, and registration date. For each user, the admin has the option to promote a student to the admin role or demote an admin back to student with a single button click. The backend validates this change, and specifically prevents an admin from accidentally removing their own admin access. Finally, the admin can browse a recent activity log that shows the last one hundred interactions across all users on the platform, populated with the corresponding user's name and email for context.

3. Non-Functional Requirements

3.1 Responsive User Interface

The entire frontend of AI Programming Tutor & Bug Explainer is built with React 18 and Vite as the build tool, styled exclusively using Tailwind CSS utility classes. The interface is designed to be fully responsive across three primary viewport sizes: desktop screens at 1024 pixels and above, tablet screens at 768 pixels, and mobile screens at 375 pixels. Every page including the dashboard, all AI tool pages, the admin panel, and the landing page adapts its layout to the available screen width without any horizontal scrolling or broken elements. For code input, the platform uses the Monaco Editor, the same editor engine that powers Visual Studio Code. This gives students a familiar, professional-grade code editing experience with syntax highlighting for dozens of programming languages, proper indentation handling, and line numbers even inside a browser. All AI responses are rendered as formatted Markdown, meaning code blocks appear with syntax highlighting and proper monospace formatting, headers are properly sized, and bold or italic text is rendered correctly. This makes the AI responses significantly easier to read than plain text.

3.2 Performance

The system is designed with performance expectations appropriate for a student-focused application that relies on external AI APIs. AI-powered endpoints are expected to respond within eight seconds under normal broadband conditions, accounting for the latency of the external AI provider. The Groq API, which is the recommended free-tier provider, typically responds in under two seconds due to its specialized inference hardware. The frontend application itself, when built for production using Vite, should complete its initial page load in under three seconds on a standard connection. The GitHub Repository Explainer should be able to fetch and render the file tree for a repository with up to five hundred files within five seconds. PDF text extraction using the unpdf library should complete within ten seconds for documents of up to twenty pages. These targets are achievable with the current technology choices and will serve the expected user load of a small-to-medium student platform without requiring caching or additional optimization in the initial version.

3.3 Security

Security is implemented at multiple layers throughout the application. At the data layer, every user password is processed through bcryptjs with a salt factor of ten before being written to MongoDB. The raw password exists in memory only for the duration of the comparison operation during login, and is never logged, transmitted, or stored in any other form. The User model's password field is excluded from all standard database queries by default and must be explicitly requested when needed, reducing the surface area for accidental password exposure. Authentication state is managed using JSON Web Tokens signed with a secret key that exists only on the server, stored as an environment variable and never included in any client-side code or frontend bundle. Tokens carry a configurable expiry of seven days by default. Every protected route passes through an auth middleware function that verifies the token's signature, checks it has not expired, retrieves the corresponding user from the database, and attaches the user object to the request. Admin-only routes perform an additional role check to ensure the authenticated user holds the admin role. All AI provider API keys including keys for Groq, Gemini, OpenAI, and GitHub are stored exclusively in server-side environment variables. The frontend client never makes direct calls to any AI provider; all AI requests go through the backend, ensuring the keys are never exposed in network traffic visible to the browser. Rate limiting via the express-rate-limit middleware is applied to all API routes to protect against abuse. In production, the CORS configuration restricts cross-origin requests to only the CLIENT_URL origin specified in the environment, blocking requests from any other domain.

3.4 Scalability and Maintainability

The application is built with scalability in mind at the architectural level. The AI provider integration is fully abstracted behind a single aiService.js file that exposes one function, askAI, to the rest of the application. Switching from Groq to Gemini or OpenAI requires only changing the AI_PROVIDER environment variable; no code changes are needed anywhere else. All AI prompt text is centralized in a separate prompts.js file, which means updating the personality, tone, or structure of any AI feature requires editing a single file rather than hunting through controllers. The backend is stateless by design; no session data is stored on the server, which means it can be scaled horizontally behind a load balancer without any session synchronization concerns.

MongoDB Atlas supports vertical and horizontal scaling through tier upgrades, and the schema is designed with indexed fields such as userId on ChatHistory for efficient querying even as the dataset grows. The codebase follows a clean separation of concerns with routes, controllers, services, models, middleware, and configuration each in their own directories, making it straightforward for a new developer to understand and extend without touching unrelated code.

4. User Roles

4.1 Student

The Student role is the default role assigned to every new user who registers on the platform. A user with the Student role has full access to all seven AI tools the AI Tutor, Bug Explainer, Compiler Error Translator, Code Improver, GitHub Repository Explainer, PDF and Image Document Explainer, and Practice Problem Generator. They can also use Voice Tutor to hear any AI response read aloud, and they can upload PDF files and images for document explanation. Students who are logged in receive additional benefits beyond what a guest can access. Their interactions are saved to a personal history that they can review at any time, showing their last fifty conversations across all tools. Every meaningful interaction earns XP, which accumulates toward higher levels displayed on their dashboard. The AI memory system quietly tracks their recent activity and weak areas, and the AI tutor uses this information to personalize its responses over time making the platform feel like it genuinely knows the student rather than treating every session as a fresh start. Even without logging in, a visitor can use all AI tools as a guest. Guest interactions are processed by the same AI backend and return the same quality of responses, but no history is saved and no XP is awarded. This design ensures that the platform is useful from the very first visit without any registration friction.

4.2 Admin

The Admin role is a privileged role that is not available through the normal registration flow. A user receives the Admin role only when their email address is present in the ADMIN_EMAILS environment variable at the time they register or log in. This means that creating the first administrator requires only adding an email to a server configuration file no database access, no manual SQL commands, and no separate admin registration page is needed. An admin user retains all the capabilities of a Student they can use every AI tool, earn XP, access their personal dashboard, and benefit from memory personalization. In addition, they have exclusive access to the Admin Panel, which gives them visibility over the entire platform. Through the Admin Panel, they can view aggregate statistics showing the total number of registered users, the total number of AI requests processed, and a breakdown of how frequently each AI tool has been used. They

can browse the full user list with profile details, and they can promote or demote any user's role with a single action. They also have access to a recent activity log showing the last one hundred interactions across all students, complete with the user's name and email, the feature used, and the timestamp giving them a live view of how the platform is being used.

5. Use Cases

Use Case 1: User Registers an Account

User Action

A new visitor arrives at the platform for the first time and decides to create an account so they can access their personal history and XP tracking. They navigate to the registration page and fill in three fields: their full name, a valid email address, and a password they choose for themselves. The password must be at least six characters long if it is shorter, the form shows a validation message and does not submit. When all three fields are filled and the password meets the minimum length requirement, the visitor clicks the Register button.

System Response

The HTTP POST request reaches the backend at /api/auth/register with the three field values in the request body. The Express middleware first checks that all three fields are non-empty — if any are missing, it returns HTTP 400 immediately with a message indicating which fields are required. If validation passes, the backend calls User.findOne({ email }) on MongoDB to check whether the submitted email address already exists in the User collection. If a document is found, the server returns HTTP 409 Conflict with the message "Email already registered" and stops there the visitor is shown a prompt to log in with that email instead. If the email is new, the next step is determining what role to assign. The server reads the ADMIN_EMAILS environment variable, splits it on commas, and checks whether the submitted email appears in the resulting list after trimming whitespace and normalizing to lowercase. If it does, the role is set to 'admin'; otherwise it is set to 'student'. The User.create() call is then made, which triggers the Mongoose pre-save hook that generates a bcrypt salt of cost factor ten and hashes the password before the document is written to MongoDB. The raw password is discarded from memory at this point. Once the document is created, jwt.sign() is called with the new user's _id and the JWT_SECRET environment variable, producing a signed token that expires in seven days. The server responds with HTTP 201 and returns the user's profile object passed through toSafeJSON() which deletes the password field

along with the token. The frontend stores the token in local state for use in future authenticated requests.

Use Case 2: User Logs In

User Action

A student who already has an account opens the login page. They type their registered email address into the email field and their password into the password field. They click the Login button to submit the form.

System Response

The POST request arrives at /api/auth/login. The backend validates that both the email and password fields are present in the request body if either is missing, HTTP 400 is returned immediately. The backend then calls User.findOne({ email }).select('+password') to retrieve the user document including the password field, which is normally excluded from query results by the schema's select: false setting. If no document is found for the submitted email, the server returns HTTP 401 with the message "Invalid email or password". The same response is returned even if the email exists but the password is wrong the two failure cases produce identical error messages deliberately, so an attacker cannot use the error response to determine which part of the credential was incorrect. If the document is found, the backend calls user.matchPassword(password), which internally calls bcrypt.compare() to verify the submitted plain-text password against the stored hash. If this returns false, HTTP 401 is returned with the same generic message. If it returns true, the backend performs one additional check: it calls isAdminEmail() with the user's email. If the email appears in the current ADMIN_EMAILS list but the user's stored role is still 'student' which can happen if the user registered before they were added to the admin list the role is updated to 'admin' and user.save() is called to write the change before responding. This synchronization step ensures that adding a new admin to the environment configuration takes effect automatically on their next login without any manual database work. A new JWT token is then signed and returned to the client along with the complete user profile.

Use Case 3: Student Asks the AI Tutor a Question

User Action

A student is studying and encounters a concept they do not understand for example, they have heard the word "recursion" in class but cannot picture what it actually means in practice, or they want to understand how a for loop works in Python and how it is different from a while loop. They navigate to the Tutor page, type their question into the input field in whatever way feels natural to them this might be in English, in Bangla, or in a mix of both and click the Ask button. The student does not need to format their question in any particular way; the system accepts it exactly as written.

System Response

The POST request arrives at /api/ai/tutor with the message field. The backend first checks whether the request carries a valid JWT token in the Authorization header. If the user is logged in, req.user is populated with the authenticated user object, and getContext(req.user._id) is called. This function runs Memory.findOne({ userId }) and WeakTopic.find({ userId }).sort({ mistakeCount: -1 }).limit(5) in parallel. It takes the last six entries from the memory document's notes array and the top five WeakTopic records and formats them into a compact context paragraph that reads like a human summary of the student's recent learning history. This memory context, along with the student's message, is passed into askAI('tutor', { message, memory }). The AI service constructs a request to the configured provider Groq, Gemini, OpenAI, or Mock with the tutor system prompt as the system message and a combined user message that starts with the memory block and ends with the student's question. The AI is instructed to respond in four parts: a one-line simple definition, a short everyday analogy drawn from real life, a tiny working code example, and a sentence of encouragement. The language of the response adapts to how the student wrote their question if they wrote in Bangla, the AI leans toward Bangla; if in English, it leans toward English; and for a Banglish mix it mirrors that style. When the AI response arrives, the backend immediately sends it back to the client as JSON so the student sees their answer without any further waiting. In the background, three parallel best-effort operations are then triggered. ChatHistory.create() saves the interaction to the database with the feature tag 'tutor' and the student's question as the message

and topic fields. addNote() writes a new memory note for example, "Asked the tutor about: recursion" into the user's Memory document, automatically discarding anything beyond the forty most recent notes using the $slice operator. And awardXP() adds five points to the user's xp field and recalculates their level. All three operations are wrapped in individual try/catch blocks so that even a complete database failure at this stage does not prevent the AI response from being delivered to the student.

Use Case 4: Student Submits Code for Bug Explanation

User Action

A student has been working on a piece of code perhaps a C program to find the sum of an array, or a Python function to count characters in a string and something is not working. The output is wrong, or the program crashes, or it behaves differently from what they expected. They navigate to the Bug Explainer page, paste their code into the Monaco editor, and optionally select the programming language from the dropdown if they want more language-specific feedback. They click the Explain button.

System Response

The POST request arrives at /api/ai/bug-explainer with the code field and optionally the language field. If no code is provided, the server returns HTTP 400 immediately. The backend retrieves the user's memory context through getContext() if the user is logged in, then calls askAI('bugExplainer', { code, language, memory }) with the bugExplainer system prompt. The AI receives the code along with the persona and rules established by the system prompt and the user's memory context. It is instructed to respond in four clearly labeled sections. In the first section, "What I see", it summarizes the core bug in a single plain-language sentence — describing the problem in terms the student can immediately grasp, without using compiler jargon. In the second section, "Line by line", it quotes or references the specific line or lines where the problem occurs and explains in simple terms exactly why that line is problematic for instance, explaining that a loop condition written as i <= arr.length causes the last iteration to access an index that does not exist in the array, and that in C this results in undefined behavior because the memory at that location belongs to something else. In the third section, "Fixed code", it presents the complete

corrected version of the student's code inside a properly formatted code block, not as a diff but as a clean, complete, runnable version. In the fourth section, "How to avoid this next time", it offers one or two short and memorable prevention habits that address the root cause rather than just the specific instance. The AI response is returned to the client immediately as JSON. In the background, ten XP is awarded to the logged-in user, a memory note is recorded for example, "Debugged some Python code" and the interaction is saved to ChatHistory under the 'bugExplainer' feature tag.

Use Case 5: Student Translates a Compiler Error Message

User Action

A student has just tried to compile or run their program and received an error message from the compiler, interpreter, or runtime environment. The message might be something like "Exception in thread main java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5" or "Segmentation fault (core dumped)" or "TypeError: 'NoneType' object is not subscriptable". To a beginner, these messages are opaque and frightening. The student copies the full error message, navigates to the Compiler Error Translator page, pastes it into the input field, optionally selects the programming language it came from, and clicks Translate.

System Response

The POST request arrives at /api/ai/error-translator with the error field containing the raw error text. The backend validates that the field is present and non-empty, retrieves the user's memory context if they are logged in, and calls askAI('errorTranslator', { error, language, memory }) with the errorTranslator system prompt. The AI is given the error text and any language context and instructed to respond in four parts, each written in plain Bangla and English together. In the first part, "Simple meaning", the AI rephrases what the error message is actually saying in one or two sentences that a first-year student can understand immediately for example, translating "ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5" into "তোমার array তে মাত্র 5টা element আছে (index 0 থেকে 4 পর্যন্ত), কিন্তু তুমি index 5 access করার চেষ্টা করেছ যেটা exist করে না।" In the second part, "Why it usually happens", the AI describes the most common

programming patterns that produce this specific error in the relevant language for the ArrayIndexOutOfBoundsException example, explaining that it almost always comes from a loop condition that uses <= instead of < when comparing against the array's length. In the third part, "How to fix it", the AI gives concrete, numbered steps the student can take right now to investigate and resolve the problem. In the fourth part, it adds a brief illustrative code snippet if one would make the explanation significantly clearer. The response is returned to the client. Five XP is awarded to the logged-in user, a memory note is recorded for example, "Hit a Java error: Exception in thread main java.lang.ArrayIndexOutOfBo..." and the interaction is saved under 'errorTranslator'.

Use Case 6: Student Requests Code Improvement Suggestions

User Action

A student has written a piece of code that works correctly it produces the right output but they have a feeling it could be written better. Perhaps it feels messy, or they used variable names like 'a', 'b', and 'x' that they know are not very descriptive, or their solution uses a nested loop and they have heard that nested loops can be slow. They navigate to the Code Improver page, paste their working code into the Monaco editor, optionally select the language, and click Improve.

System Response

The POST request arrives at /api/ai/code-improver. The backend validates that the code field is present, retrieves the memory context, and calls askAI('codeImprover', { code, language, memory }) with the codeImprover system prompt. The AI is instructed to review the code with the perspective of an encouraging senior developer talking to a junior, not a critic grading an assignment. It is explicitly told not to rewrite the code into something the student would not recognize the goal is to suggest improvements that are close enough to the student's own approach that they can follow along and learn from each change. The response is structured in three parts. The first part, "What's good", genuinely identifies one specific positive aspect of the code not a generic compliment, but something concrete, such as noting that the student correctly initialized their counter variable before the loop, or that their logic for handling the edge case of an empty input is correct. This part exists because students need to know what they got right, not just what

to fix. The second part, "Suggestions", covers specific improvements with a one-to-two sentence explanation for each for example, suggesting that 'a' should be renamed 'totalSum' to make its purpose immediately clear, or explaining that the nested loop can be replaced with a dictionary lookup so that the algorithm runs in O(n) time instead of $O(n^2)$, and briefly explaining what O(n) means in practical terms. The third part, "Improved version", shows a focused code snippet that applies one or two of the most impactful suggestions so the student can see the result concretely. Ten XP is awarded to the logged-in user and the interaction is saved under 'codeImprover'.

Use Case 7: Student Explores a GitHub Repository

User Action

A student is browsing GitHub and finds a project that interests them perhaps a simple web app, a library, or a beginner project shared by another student. They want to understand what it does, how it is structured, and where to start reading. They copy the repository URL, navigate to the GitHub Repository Explainer page, paste the URL into the input field, select their preferred output language Bangla, Banglish, or English and click Explain.

System Response

The POST request arrives at /api/repo/overview. The backend calls parseRepoUrl() to extract the owner and repository name from the submitted URL. It then calls getRepoSnapshot(), which internally makes three API calls to the GitHub REST API: a GET to /repos/{owner}/{repo} for the repository's metadata including its name, description, primary language, star count, and default branch; a GET to /repos/{owner}/{repo}/readme to fetch the README file content, which is returned as a base64-encoded string that the service decodes to plain text; and a GET to /repos/{owner}/{repo}/git/trees/{branch}?recursive=1 to retrieve the complete recursive file tree, which returns every file and directory path in the repository as a flat array. All three results are assembled into a prompt for the askAI('repoOverview', ...) call. The language directive based on the student's selection is prepended to the prompt. The AI is instructed to respond in five labeled sections using the Bangla headings defined in the system prompt, and is strictly told to base its response only on the data provided not to invent files, features, or technologies that are not visible in the README or file tree. The completed explanation along with the meta object and full file

tree array are returned to the client. The frontend renders the file tree as a navigable list of clickable file names alongside the AI's overview explanation. When the student clicks on any file name in the rendered tree, the frontend sends a POST to /api/repo/file with the repository URL, the selected file path, and the default branch name. The backend calls parseRepoUrl() again to get the owner and repo, confirms the branch, and calls getFileContent() to fetch the raw file content from the GitHub API. This content which might be source code, a configuration file, a Markdown document, or any other text file is sent to askAI('fileExplain', { path, content, lang }) with the fileExplain system prompt, which asks the AI to explain what this specific file is responsible for in the overall project, walk through its most important sections by referencing function or block names, and explain any tricky concepts it introduces. The explanation is returned to the client and displayed alongside the file content.

Use Case 8: Student Uploads a Document for Explanation

User Action

A student has a PDF of their lecture slides from class, or a photo they took of their professor's whiteboard, or a scanned image of a textbook page they want to understand better. They navigate to the Document Explainer page, click the upload area, and select the file from their device. They choose their preferred explanation language. Optionally, they type a specific question they have about the content for example, "what does the recursion example on page 3 actually do?" into the question field. They click Explain.

System Response

The POST request arrives at /api/doc/explain as a multipart/form-data request. The Multer middleware configured for this route intercepts the file and holds it in memory as a Buffer object it is never written to the server's filesystem. The backend reads the file's MIME type from req.file.mimetype to determine which processing path to follow. If the MIME type is application/pdf, the backend passes new Uint8Array(file.buffer) into getDocumentProxy() from the unpdf library. This creates an in-memory PDF document proxy without any filesystem interaction. It then calls extractText(pdf, { mergePages: true }), which reads every page and concatenates all text content into a single string. If extraction produces an empty string which

happens when the PDF is a scanned image rather than a text-based document the server returns HTTP 422 with a message advising the student to upload a photo of the page instead, because the image processing path can handle visually encoded content. If text is successfully extracted, it is checked against the sixteen-thousand-character limit. If it exceeds that limit, it is truncated at exactly that point with a note "... (truncated) ..." appended so the AI knows the document continued beyond what it received. The extracted text is then passed into askAI('docExplain', { lang, question, text }). If the MIME type begins with image/, the backend converts the file buffer to a base64 string using file.buffer.toString('base64') and calls askAIVision('docExplain', { lang, question }, { base64: ..., mimeType: mime }). This function constructs the message payload in the multimodal format required by vision-capable AI models, embedding the image data directly in the message content. The AI receives the image alongside the docExplain system prompt, which instructs it to identify what the document contains, explain the key ideas simply with examples where helpful, identify and explain any code found in the material, and if a specific student question was provided, answer that question first before giving the general explanation. The completed explanation is returned to the client along with the filename and the kind of file that was processed.

Use Case 9: Student Listens to an AI Response with Voice Tutor

User Action

A student has received an AI explanation of a concept they were studying perhaps a tutor response about how recursion works, or a bug explanation for a piece of code they submitted. They have read through the response once, but they learn better by hearing information rather than reading it, or they want to listen while they follow along in their code editor without switching focus back and forth. They click the speaker icon or the Voice button that appears alongside the AI response text.

System Response

The click event handler in the client-side React component creates a new SpeechSynthesisUtterance object and assigns it the full text content of the AI response. It sets the lang property of the utterance to the appropriate language locale based on the response language for a Banglish or Bangla response this would typically be set to 'bn-BD' or a suitable fallback, and

for an English response to 'en-US'. It then calls window.speechSynthesis.speak(utterance) to hand the utterance to the browser's built-in speech synthesis engine. The browser generates audio from the text using the device's available voice data and plays it through the default audio output speakers, headphones, or whatever the operating system has configured. The entire operation happens entirely within the browser with no network request, no API call, and no server involvement of any kind. There is no check against the user's account tier, no authentication token verified, and no difference in behavior between a guest visitor and a logged-in student the feature works identically for both. The Web Speech API is supported natively in Chrome, Firefox, Safari, and Edge on both desktop and mobile operating systems without requiring any plugin or extension.

Use Case 10: Admin Views Statistics and Manages Users

User Action

An administrator wants to check how the platform is being used how many students have registered, which AI tools are most popular, and which students are the most active. They also notice that a particular student should be given admin access so they can help monitor the platform. The admin logs in with their admin-role account, navigates to the Admin Panel by visiting the /admin route, reviews the statistics and user list, and clicks the Promote button next to the student's name.

System Response

When the admin navigates to /admin, the frontend first confirms that the authenticated user has the admin role if not, the route redirects them away immediately. Once on the panel, the frontend sends a GET request to /api/admin/stats. On the server, the protect middleware verifies the JWT token and the admin middleware checks that req.user.role 'admin' before the handler executes. The stats handler then calls Promise.all() with five parallel database operations: User.countDocuments() for total users, User.countDocuments({ role: 'admin' }) for admin count, ChatHistory.countDocuments() for total AI interactions ever recorded, a ChatHistory.aggregate() pipeline that groups all records by the feature field and produces a count for each group, and User.find().sort({ xp: -1 }).limit(5).select('name xp level') for the top-five leaderboard. All five results are returned in a single JSON response and rendered on the admin panel as statistics cards

showing total users, total admins, and total requests alongside a chart showing the breakdown of usage by feature and a leaderboard of the most active students. The admin also sees the full user list, loaded by a separate GET to /api/admin/users, which returns up to two hundred user documents sorted by registration date descending, each with the password field removed. When the admin clicks the Promote button next to a specific student, the frontend sends a PATCH request to /api/admin/users/:id/role with the JSON body { "role": "admin" }. The server validates that the role value is either 'student' or 'admin', checks that the target user's _id does not match the authenticated admin's own _id with a role of non-admin which would be a self-lockout and calls User.findByIdAndUpdate(req.params.id, { role }, { new: true }) to update the document in MongoDB. The updated user object is returned and the frontend refreshes the user list to reflect the new role immediately.